

强化学习在推荐系统中的应用综述

1 强化学习在推荐系统中的典型应用方式

强化学习应用于推荐系统时，通常并不会完全改变工业推荐系统中“召回-排序-重排-展示”的整体链路。在大规模物品库场景下，系统仍然需要先通过多路召回或向量检索得到候选集，再在候选集内进行打分、排序或采样。强化学习与传统监督学习推荐的核心区别主要体现在训练目标上：传统推荐通常学习点击率、转化率或观看时长等即时反馈预测模型，而强化学习方法则将推荐建模为序贯决策问题，通过策略或价值函数优化长期收益。典型方法可以分为上下文老虎机、policy-based 方法、value-based 方法以及 Actor-Critic 方法。

1.1 上下文老虎机方法

上下文老虎机可以看作推荐系统中最简单的一类强化学习方法。在该建模方式下，系统在每一轮推荐时观察用户上下文和候选物品集合，然后从候选集合中选择一个或多个物品展示给用户，并根据用户反馈获得奖励。与完整强化学习不同，上下文老虎机通常不显式建模状态转移，即主要关注当前推荐动作带来的即时收益。

设第 t 轮用户状态或上下文为 s_t ，候选物品集合为 $\mathcal{A}_t$ ，系统选择动作 $a_t \in \mathcal{A}_t$ ，并观察奖励 r_t 。上下文老虎机的目标是学习一个选择规则：

$$a_t = \arg \max_{a \in \mathcal{A}_t} \text{score}(s_t, a).$$

以 LinUCB 为例，算法对每个候选物品计算：

$$p_{t,a} = x_{t,a}^\top \hat{\theta}_a + \alpha \sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}},$$

其中 $x_{t,a}$ 表示用户-物品上下文特征，第一项 $x_{t,a}^\top \hat{\theta}_a$ 表示当前模型对收益的估计，第二项表示不确定性奖励。当某个物品或某类上下文样本较少时， $\sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}}$ 较大，算法会更倾向于探索该物品。因此，上下文老虎机的核心思想是：

推荐分数 = 收益估计 + 不确定性奖励。

在动作选择上，上下文老虎机一般不会在全量物品库中直接搜索，而是在当前候选池中计算每个候选物品的 UCB 分数或 Thompson Sampling 分数，然后选择分数最高的物品。因此其工程形态仍然类似传统推荐：

候选召回 $\rightarrow$ 候选集内打分 $\rightarrow$ 选择 Top- K 。

1.3 Value-based 方法

Value-based 方法不直接学习动作概率，而是学习动作价值函数：

$$Q(s, a),$$

表示在用户状态 s 下推荐物品 a 后能够获得的长期累计收益。传统推荐系统中的排序分数通常是点击率、转化率或观看时长等即时指标，例如：

$$\text{score}(s, a) = \text{pCTR}(s, a),$$

而 value-based 强化学习方法将排序分数替换为长期价值：

$$\text{score}(s, a) = Q(s, a).$$

推荐时选择：

$$a^* = \arg \max_{a \in \mathcal{A}} Q(s, a).$$

如果考虑状态转移，Q-learning 的目标为：

$$y_t = r_t + \gamma \max_{a'} Q_{\bar{\phi}}(s_{t+1}, a'),$$

Critic 或 Q 网络通过最小化 TD 误差进行更新：

$$\mathcal{L}_Q = [Q_{\phi}(s_t, a_t) - y_t]^2.$$

该方法的优势是能够把当前推荐动作对未来用户行为的影响纳入价值估计，例如后续点击、连续观看、复购、留存等长期收益。

然而，推荐系统的动作空间通常极大，直接计算

$$\max_{a \in \mathcal{A}} Q(s, a)$$

并不可行。因此 value-based 方法通常仍然依赖候选召回：

$$\mathcal{A} \rightarrow C \rightarrow \text{TopK}_{a \in C} Q(s, a).$$

也就是说，系统先通过双塔召回、协同过滤、热门召回等方式得到候选集 C ，再在候选集内用 $Q(s, a)$ 进行排序。

对于列表推荐，动作可以是整个 slate：

$$A = (a_1, a_2, \dots, a_K).$$

此时价值函数为：

$$Q(s, A).$$

但由于列表动作空间呈组合爆炸，通常不会直接枚举所有列表。以 SlateQ 思路为例，可以将列表价值分解为 item-level 长期价值与用户选择概率的组合：

$$Q(s, A) = \sum_{a \in A} P(a | s, A) Q(s, a),$$

其中 $P(a | s, A)$ 表示用户在列表 A 中选择物品 a 的概率。这种方式避免了直接学习每个列表动作的价值，使得列表推荐中的长期价值优化更容易落地。

1.4 Actor-Critic 方法在推荐系统中的应用

Actor-Critic 方法可以看作 policy-based 与 value-based 方法的结合。其中 Actor 负责生成推荐动作，Critic 负责评价该动作的长期价值。在推荐系统中，Actor-Critic 的基本结构为：

$$s_t = E_\psi(h_t),$$

$$a_t = \mu_\theta(s_t),$$

$$Q_\phi(s_t, a_t),$$

其中 h_t 表示用户历史行为序列， E_ψ 是状态编码器， μ_θ 是 Actor， Q_ϕ 是 Critic。Actor 根据用户状态生成动作，Critic 判断该动作在当前状态下的长期收益。

在大规模推荐场景中，Actor 往往不会直接输出离散 item ID，而是输出一个连续动作向量：

$$\hat{a}_t = \mu_\theta(s_t) \in \mathbb{R}^d.$$

该向量可以理解为当前用户的“理想物品表示”或推荐方向。随后系统通过 ANN 或 KNN 在物品 embedding 库中找到与 $\hat{a}_t$ 最接近的一批真实物品：

$$C_t = \text{ANN}(\hat{a}_t, \{v_i\}_{i=1}^{|\mathcal{I}|}).$$

然后 Critic 在候选集内重新评价：

$$a_t^* = \arg \max_{a \in C_t} Q_\phi(s_t, v_a).$$

若需要推荐 Top-K，则选择：

$$A_t = \text{TopK}_{a \in C_t} Q_\phi(s_t, v_a).$$

这种方式本质上属于确定性策略梯度方法。Actor 给定状态后直接输出动作向量：

$$\hat{a}_t = \mu_\theta(s_t),$$

其优化目标是最大化 Critic 对该动作的价值估计：

$$J(\theta) = \mathbb{E}_s [Q_\phi(s, \mu_\theta(s))].$$

因此 Actor loss 可以写为：

$$\mathcal{L}_{actor} = -\mathbb{E}_s [Q_\phi(s, \mu_\theta(s))].$$

对应的确定性策略梯度为：

$$\nabla_\theta J(\theta) = \mathbb{E}_s \left[\nabla_a Q_\phi(s, a) \Big|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s) \right].$$

直观来说，Critic 告诉 Actor 动作向量往哪个方向移动能够提升长期价值，Actor 则根据该梯度调整自己的参数。

Critic 的更新通常采用 TD 目标。对于日志样本

$$(s_t, i_t, r_t, s_{t+1}),$$

其中 i_t 是历史曝光物品，其 embedding 为 v_{i_t} 。Critic 输入为：

$$Q_\phi(s_t, v_{i_t}).$$

利用 target Actor 和 target Critic 构造目标：

$$y_t = r_t + \gamma Q_{\bar{\phi}}(s_{t+1}, \mu_{\bar{\theta}}(s_{t+1})),$$

Critic loss 为：

$$\mathcal{L}_{critic} = [Q_\phi(s_t, v_{i_t}) - y_t]^2.$$

其中 $\bar{\theta}$ 和 $\bar{\phi}$ 表示 target network 参数，用于稳定训练。

在该框架下，用户状态 embedding 通常不是固定向量，而是由状态编码器生成：

$$s_t = E_\psi(h_t).$$

因此，Actor loss 和 Critic loss 都可以反向传播到状态编码器，从而更新 user/state embedding 的生成参数。item embedding v_i 则有两种处理方式：一种是先通过传统推荐、自监督或双塔召回任务预训练后固定；另一种是将其作为可训练参数，通过 Critic loss 或辅助监督任务进行更新。需要注意的是，ANN/KNN 将连续动作向量映射到真实物品的过程不可导，因此梯度不会从最近邻检索步骤本身反向传播。训练时真正可导的是连续空间中的 $Q_\phi(s, \mu_\theta(s))$ 以及日志物品上的 $Q_\phi(s, v_i)$ 。

由于推荐系统训练数据通常来自历史策略，Actor-Critic 推荐模型还需要处理离线训练中的分布偏移问题。如果 Actor 输出的动作远离历史日志分布，Critic 对这些动作的价值估计可能不可靠。因此实际系统中常加入行为克隆或监督约束：

$$\mathcal{L}_{BC} = \|\mu_\theta(s_t) - v_{i_t}\|^2,$$

最终 Actor loss 可以写成：

$$\mathcal{L}_{actor} = -Q_\phi(s_t, \mu_\theta(s_t)) + \lambda \|\mu_\theta(s_t) - v_{i_t}\|^2.$$

该约束的作用是：一方面让 Actor 朝着高长期价值的方向更新，另一方面避免 Actor 产生过多历史日志中未覆盖的分布外动作。

总体来看，Actor-Critic 在推荐系统中的作用可以概括为：Actor 负责根据用户状态生成推荐方向或候选动作，Critic 负责评估动作的长期价值，Actor 再根据 Critic 的梯度进行更新。在工程实现中，连续动作向量、ANN 候选检索、Critic 重排以及行为约束通常是这类方法能够落地的关键。

2 SlateQ 算法

SlateQ 是一种面向推荐列表场景的强化学习算法，其核心目标是解决推荐系统中 **组合动作空间过大** 的问题。在传统强化学习中，智能体在状态 s 下选择一个动作 a ，并通过动作价值函数 $Q(s, a)$ 评估该动作的长期收益。然而，在推荐系统中，系统通常不是向用户推荐单个物

品，而是一次展示一个由多个物品组成的推荐集合，即 slate。若候选物品集合为 $\mathcal{I}$ ，每次推荐 k 个物品，则一个推荐动作可以表示为

$$A = \{i_1, i_2, \dots, i_k\}, \quad A \subseteq \mathcal{I}, \quad |A| = k.$$

此时动作空间规模为

$$\binom{|\mathcal{I}|}{k},$$

当候选物品数量很大时，直接对每一个推荐集合学习 $Q(s, A)$ 几乎不可行。SlateQ 的基本思想是：不直接学习整个推荐列表的价值，而是将推荐列表的长期价值分解为用户可能选择的单个物品的长期价值，再通过用户选择模型组合得到整个 slate 的价值。

2.1 推荐列表强化学习建模

在 SlateQ 中，推荐过程可以建模为一个马尔可夫决策过程。状态 s_t 表示用户在时刻 t 的兴趣状态、历史行为、上下文信息等；动作 A_t 表示系统展示给用户的推荐列表；用户看到推荐列表后，会从列表中选择某个物品 $i \in A_t$ ，也可能不选择任何物品，记为 $\perp$ 。交互之后，系统得到即时奖励 r_t ，并转移到下一状态 s_{t+1} 。推荐系统的优化目标是最大化长期折扣回报：

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t r_t \right],$$

其中 $\gamma \in [0, 1]$ 为折扣因子， π 为推荐策略。

如果直接将整个推荐列表 A 作为动作，则动作价值函数为

$$Q^\pi(s, A) = R(s, A) + \gamma \sum_{s'} P(s' | s, A) V^\pi(s'),$$

其中 $R(s, A)$ 表示在状态 s 下展示列表 A 的即时奖励期望， $P(s' | s, A)$ 表示展示列表 A 后转移到下一状态 s' 的概率， $V^\pi(s')$ 表示后续状态价值。该形式虽然符合标准强化学习定义，但由于 A 是组合动作，直接学习和优化 $Q(s, A)$ 的代价极高。

2.2 SlateQ 的核心假设

SlateQ 能够进行价值分解，依赖于一个关键假设，即 **奖励和状态转移主要依赖于用户最终选择的物品**。也就是说，用户看到推荐列表 A 后，虽然系统展示了多个物品，但真正影响即时奖励和后续状态的，主要是用户最终点击、观看或消费的那个物品。

设用户在状态 s 下看到列表 A 后选择物品 i 的概率为

$$P(i | s, A),$$

其中 $i \in A$ 。若用户没有选择任何物品，则记为 $\perp$ 。在该假设下，列表级即时奖励可以写成

$$R(s, A) = \sum_{i \in A} P(i | s, A) R(s, i),$$

其中 $R(s, i)$ 表示用户选择物品 i 后产生的即时奖励期望。类似地，状态转移概率可以写成

$$P(s' | s, A) = \sum_{i \in A} P(i | s, A) P(s' | s, i).$$

这意味着整个推荐列表的影响可以被表示为不同用户选择结果的加权平均。

2.3 SlateQ 的价值分解

根据上述假设，列表动作价值函数可以展开为

$$Q^\pi(s, A) = R(s, A) + \gamma \sum_{s'} P(s' | s, A) V^\pi(s').$$

将奖励分解和状态转移分解代入上式，有

$$Q^\pi(s, A) = \sum_{i \in A} P(i | s, A) R(s, i) + \gamma \sum_{s'} \sum_{i \in A} P(i | s, A) P(s' | s, i) V^\pi(s').$$

交换求和顺序后可得

$$Q^\pi(s, A) = \sum_{i \in A} P(i | s, A) \left[R(s, i) + \gamma \sum_{s'} P(s' | s, i) V^\pi(s') \right].$$

定义物品级长期价值函数为

$$Q^\pi(s, i) = R(s, i) + \gamma \sum_{s'} P(s' | s, i) V^\pi(s'),$$

则列表级价值函数可以分解为

$$Q^\pi(s, A) = \sum_{i \in A} P(i | s, A) Q^\pi(s, i).$$

该公式是 SlateQ 的核心。它表明，推荐列表的长期价值不是列表中各个物品价值的简单相加，而是由用户选择概率加权后的期望长期价值。

从直观上看， $Q^\pi(s, i)$ 表示用户真正选择并消费物品 i 后带来的长期价值，而 $P(i | s, A)$ 表示物品 i 在当前列表中被用户选择的概率。因此，一个物品即使长期价值很高，如果用户几乎不会点击它，它对当前推荐列表的贡献也可能很小；相反，一个物品即使点击概率很高，但长期价值较低，也可能会抢占其他高价值物品的点击概率，从而降低整体长期收益。

2.4 用户选择模型

为了计算

$$P(i | s, A),$$

SlateQ 需要引入用户选择模型。常用的一类选择模型是多项 Logit 模型，即 MNL 模型。设物品 i 在状态 s 下的吸引力为

$$v(s, i) > 0,$$

用户不选择任何物品的吸引力为

$$v(s, \perp) > 0.$$

那么用户选择物品 i 的概率可以写为

$$P(i | s, A) = \frac{v(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

其中分母

$$v(s, \perp) + \sum_{j \in A} v(s, j)$$

表示用户所有可选项的总吸引力，包括不点击任何物品以及点击推荐列表中的某个物品。因此，该分母本质上是一个归一化项，用来保证所有选择概率之和为 1：

$$P(\perp | s, A) + \sum_{i \in A} P(i | s, A) = 1.$$

其中

$$P(\perp | s, A) = \frac{v(s, \perp)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

将 MNL 选择模型代入 SlateQ 的价值分解式，可以得到

$$Q(s, A) = \sum_{i \in A} \frac{v(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)} Q(s, i).$$

由于分母对于列表中的所有物品是相同的，因此可以写成

$$Q(s, A) = \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

该式说明，SlateQ 优化的并不是单纯的点击率，也不是单纯的物品长期价值，而是点击概率和点击后长期价值共同决定的期望收益。

2.5 Slate 优化问题

在得到物品级价值函数 $Q(s, i)$ 和用户选择模型 $P(i | s, A)$ 后，推荐系统需要在候选物品集合 $\mathcal{I}$ 中选择一个大小为 k 的推荐列表 A ，使得列表级价值最大：

$$A^* = \arg \max_{A \subseteq \mathcal{I}, |A|=k} Q(s, A).$$

在 MNL 选择模型下，该问题可以写成

$$A^* = \arg \max_{A \subseteq \mathcal{I}, |A|=k} \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

为了将该问题写成规划形式，可以为每个候选物品 $i \in \mathcal{I}$ 引入二元变量

$$x_i = \begin{cases} 1, & \text{物品 } i \text{ 被选入推荐列表,} \\ 0, & \text{物品 } i \text{ 未被选入推荐列表.} \end{cases}$$

则推荐列表大小约束为

$$\sum_{i \in \mathcal{I}} x_i = k.$$

目标函数可以写成

$$\max_x \frac{\sum_{i \in \mathcal{I}} x_i v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in \mathcal{I}} x_j v(s, j)},$$

约束条件为

$$\sum_{i \in \mathcal{I}} x_i = k, \quad x_i \in \{0, 1\}, \quad \forall i \in \mathcal{I}.$$

这是一个分式整数规划问题。其困难在于，选择某个物品不仅会增加分子中的长期价值贡献，同时也会增加分母中的总吸引力，从而改变列表中其他物品的选择概率。因此，SlateQ 中的列表优化并不能简单退化为选择 $Q(s, i)$ 最大的 k 个物品。

在一些条件下，可以将二元变量松弛为连续变量：

$$0 \leq x_i \leq 1,$$

从而得到分式线性规划：

$$\begin{aligned} & \max_x \frac{\sum_{i \in \mathcal{I}} x_i v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in \mathcal{I}} x_j v(s, j)}, \\ & \text{s.t.} \quad \sum_{i \in \mathcal{I}} x_i = k, \quad 0 \leq x_i \leq 1. \end{aligned}$$

该松弛形式可以降低求解难度，并为大规模推荐系统中的近似 slate 优化提供基础。

2.6 SlateQ 的训练过程

SlateQ 的训练数据通常来自用户与推荐系统的历史交互。每条样本可以表示为

$$(s_t, A_t, c_t, r_t, s_{t+1}),$$

其中 s_t 为当前用户状态， A_t 为系统展示的推荐列表， c_t 为用户最终选择的物品， r_t 为即时奖励， s_{t+1} 为下一状态。如果用户没有点击或消费任何物品，则有 $c_t = \perp$ 。

SlateQ 不直接更新整个推荐列表的价值 $Q(s_t, A_t)$ ，而是更新用户实际选择的物品对应的物品级价值 $Q(s_t, c_t)$ 。对于用户选择了物品 c_t 的样本，TD 目标可以写成

$$y_t = r_t + \gamma \max_{A' \subseteq \mathcal{I}, |A'|=k} Q(s_{t+1}, A').$$

根据 SlateQ 的价值分解，下一状态下的最优列表价值为

$$\max_{A' \subseteq \mathcal{I}, |A'|=k} Q(s_{t+1}, A') = \max_{A' \subseteq \mathcal{I}, |A'|=k} \sum_{i \in A'} P(i | s_{t+1}, A') Q(s_{t+1}, i).$$

若采用 MNL 选择模型，则可以进一步写为

$$\max_{A' \subseteq \mathcal{I}, |A'|=k} \frac{\sum_{i \in A'} v(s_{t+1}, i) Q(s_{t+1}, i)}{v(s_{t+1}, \perp) + \sum_{j \in A'} v(s_{t+1}, j)}.$$

设物品级价值函数由参数为 θ 的模型 $Q_\theta(s, i)$ 近似, 则训练损失可以写成

$$\mathcal{L}(\theta) = (Q_\theta(s_t, c_t) - y_t)^2.$$

在深度强化学习实现中, 通常还会使用目标网络来稳定训练。若目标网络参数为 θ^- , 则 TD 目标可写为

$$y_t = r_t + \gamma Q_{\theta^-}(s_{t+1}, A_{t+1}^*),$$

其中

$$A_{t+1}^* = \arg \max_{A' \subseteq \mathcal{I}, |A'|=k} Q_{\theta^-}(s_{t+1}, A').$$

2.7 算法流程

SlateQ 的整体流程可以概括如下。

1. 收集用户交互数据, 得到样本

$$(s_t, A_t, c_t, r_t, s_{t+1}).$$

2. 根据历史曝光和点击数据, 学习或估计用户选择模型

$$P(i \mid s, A).$$

在 MNL 模型下, 需要估计每个物品的吸引力函数 $v(s, i)$ 。

3. 使用神经网络或其他函数近似器学习物品级长期价值函数

$$Q_\theta(s, i).$$

4. 对于下一状态 s_{t+1} , 根据当前的物品级价值函数和用户选择模型, 求解最优推荐列表

$$A_{t+1}^* = \arg \max_{A' \subseteq \mathcal{I}, |A'|=k} \sum_{i \in A'} P(i \mid s_{t+1}, A') Q_\theta(s_{t+1}, i).$$

5. 构造 TD 目标

$$y_t = r_t + \gamma \sum_{i \in A_{t+1}^*} P(i \mid s_{t+1}, A_{t+1}^*) Q_\theta(s_{t+1}, i).$$

6. 最小化 TD 误差

$$\mathcal{L}(\theta) = (Q_\theta(s_t, c_t) - y_t)^2.$$

7. 重复上述过程, 直到物品级价值函数和推荐策略收敛。

2.8 SlateQ 与传统推荐排序的区别

传统推荐排序方法通常学习的是短期反馈，例如点击率、转化率或观看时长。其目标常写为

$$\hat{y}(s, i) = P(\text{click} \mid s, i),$$

然后按照预测分数对物品排序。该方法更关注当前推荐带来的即时收益。

SlateQ 则关注长期价值，其物品级价值函数为

$$Q(s, i) = R(s, i) + \gamma \sum_{s'} P(s' \mid s, i) V(s').$$

因此，SlateQ 关心的是用户消费某个物品之后对未来收益的影响。一个物品即使短期点击率较高，如果它会导致用户后续活跃度下降，其长期价值也可能较低；反之，一个物品短期点击率不一定最高，但如果能够提升用户长期兴趣和留存，则可能具有更高的 Q 值。

同时，SlateQ 显式考虑了推荐列表内部的竞争关系。在 MNL 模型下，加入一个物品会改变分母

$$v(s, \perp) + \sum_{j \in A} v(s, j),$$

从而影响列表中其他物品的选择概率。因此，SlateQ 不是简单地选择单物品分数最高的 k 个物品，而是优化整个列表的期望长期价值。

2.9 SlateQ 的优点与局限

SlateQ 的主要优点包括以下几个方面。

首先，SlateQ 通过价值分解避免了直接学习组合动作价值函数。原始的 slate 动作空间规模为

$$\binom{|\mathcal{I}|}{k},$$

而 SlateQ 只需要学习物品级价值函数 $Q(s, i)$ ，显著降低了动作空间复杂度。

其次，SlateQ 将强化学习中的长期收益引入推荐列表优化。相比于只优化点击率或转化率的监督学习排序模型，SlateQ 能够考虑当前推荐对用户未来行为、长期活跃度和累计收益的影响。

再次，SlateQ 显式引入用户选择模型，使得推荐列表中不同物品之间的竞争关系能够被建模。这比简单的逐物品打分排序更符合实际推荐场景。

但是，SlateQ 也存在一些局限。

第一，SlateQ 依赖用户选择模型的准确性。如果 $P(i \mid s, A)$ 估计不准确，则列表级价值 $Q(s, A)$ 的计算也会产生偏差。

第二，SlateQ 依赖奖励和状态转移主要由用户最终选择物品决定的假设。在真实推荐系统中，即使用户没有点击某些曝光物品，这些物品也可能影响用户体验和后续行为。因此，当未点击曝光也会影响用户状态时，SlateQ 的分解假设可能不完全成立。

第三，SlateQ 更适合单次选择或单次消费场景。如果用户会在同一个推荐列表中连续点击多个物品，或者列表的多样性、新颖性、重复性本身会显著影响用户体验，则需要对 SlateQ 进行扩展。

第四，SlateQ 仍然面临离线强化学习中的常见问题，例如分布偏移、价值高估、反事实评估困难等。因此，在实际应用中通常需要结合离线评估、保守价值估计和在线 A/B 实验进行验证。

2.10 总结

综上，SlateQ 是一种面向推荐列表优化的价值分解型强化学习算法。它的核心思想可以概括为

$$\text{SlateQ} = \text{物品级长期价值学习} + \text{用户选择模型} + \text{推荐列表优化}.$$

具体而言，SlateQ 不直接学习整个推荐列表的动作价值 $Q(s, A)$ ，而是在用户选择模型 $P(i | s, A)$ 的帮助下，将列表价值分解为

$$Q(s, A) = \sum_{i \in A} P(i | s, A) Q(s, i).$$

其中， $Q(s, i)$ 表示用户选择并消费物品 i 后产生的长期价值， $P(i | s, A)$ 表示物品 i 在当前推荐列表中被用户选择的概率。在 MNL 选择模型下，列表优化进一步转化为

$$\max_{A: |A|=k} \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

因此，SlateQ 既避免了直接处理巨大组合动作空间的困难，又能够在推荐排序中引入长期收益优化思想，是强化学习应用于推荐系统中 slate recommendation 问题的一种典型范式。

3 基于 MDP 的长期策略学习范式：以 Top-K Off-Policy Correction for REINFORCE 为例

基于 MDP 的长期策略学习范式，是将推荐系统建模为一个序列决策问题。与传统 CTR/CVR 排序模型只关注当前曝光下的点击或转化不同，MDP 强化学习更关心当前推荐动作对后续用户状态和长期收益的影响。在该范式下，推荐系统不再只是学习一个静态打分函数，而是学习一个策略 π ，使其在连续交互过程中最大化累计回报。

Chen 等人在 *Top-K Off-Policy Correction for a REINFORCE Recommender System* 中，将 YouTube 推荐系统中的 top- K 推荐问题建模为强化学习问题，并使用 REINFORCE 进行策略梯度优化 [?]. 该工作是基于 MDP 的推荐策略学习的典型代表，其重点不是学习单个物品的监督排序分数，而是直接学习一个能够产生推荐动作的策略网络。

3.1 MDP 建模

该方法将推荐过程建模为一个马尔可夫决策过程：

$$(\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma).$$

其中, $\mathcal{S}$ 表示用户状态空间。在推荐系统中, 状态 s_t 通常由用户历史行为、上下文信息和当前兴趣表示构成。例如, 在视频推荐场景中, 用户最近观看过的视频、点击行为、观看时长、搜索行为和上下文特征都可以用于构造当前状态。论文中使用循环神经网络对用户历史交互序列进行编码, 得到一个连续向量形式的用户状态表示。

$\mathcal{A}$ 表示动作空间。在该工作中, 动作不是连续控制变量, 而是推荐系统中的候选视频或物品。因此动作空间是一个极大的离散集合:

$$a_t \in \mathcal{A}.$$

工业推荐系统中的候选物品规模可以达到百万级甚至更大, 这也是该问题相较于普通强化学习任务更加困难的原因。

$P(s_{t+1} | s_t, a_t)$ 表示状态转移概率。它描述系统在状态 s_t 下推荐物品 a_t 后, 用户状态如何变化。例如用户点击、观看、跳出、继续浏览等行为, 都会影响下一时刻的用户状态 s_{t+1} 。在真实推荐系统中, 状态转移函数通常未知, 因此该工作并不显式学习环境转移模型, 而是直接基于日志数据使用策略梯度方法学习推荐策略。

$R(s_t, a_t)$ 表示即时奖励函数。在视频推荐场景中, 奖励可以由点击、观看时长、满意度、互动行为等隐式反馈构成。论文中强调推荐系统拥有大量日志反馈, 例如 clicks 和 dwell time, 这些反馈可以作为策略学习的训练信号。因此, 奖励 r_t 可以理解为用户对当前推荐物品的即时反馈。

ρ_0 表示初始状态分布, γ 表示折扣因子。策略学习的目标是最大化长期累计回报:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)],$$

其中轨迹为

$$\tau = (s_0, a_0, s_1, a_1, \dots),$$

轨迹回报为

$$R(\tau) = \sum_{t=0}^{|\tau|} r(s_t, a_t).$$

更一般地, 也可以使用折扣回报:

$$R_t = \sum_{t'=t}^{|\tau|} \gamma^{t'-t} r(s_{t'}, a_{t'}).$$

因此, 该建模方式的关键在于:

当前推荐动作 $a_t \rightarrow$ 当前奖励 $r_t +$ 未来状态价值。

也就是说, 策略不是只学习当前哪个视频最容易被点击, 而是学习当前推荐哪个视频能够带来更好的长期用户反馈。

3.2 策略网络建模

该工作采用参数化策略 $\pi_\theta(a | s)$ 。给定用户状态 s ，策略输出每个候选物品被推荐的概率：

$$\pi_\theta(a | s).$$

在实现上，用户状态 s 可以由历史行为序列编码得到，物品 a 也有对应的 embedding 表示。策略网络通过用户状态向量和物品 embedding 的匹配程度计算物品得分，再通过 softmax 得到动作分布：

$$\pi_\theta(a | s) = \frac{\exp(s^T v_a / T)}{\sum_{a' \in \mathcal{A}} \exp(s^T v_{a'} / T)},$$

其中 v_a 表示物品 a 的动作 embedding， T 为温度系数。

由于工业推荐系统中的动作空间极大，完整 softmax 的计算代价很高。因此实际训练中通常需要使用 sampled softmax 或候选采样方法，避免对全量百万级物品计算归一化概率。

需要注意的是，虽然物品可以用连续 embedding 表示，但动作本身仍然是离散动作。也就是说，系统最终选择的是某一个具体视频或物品，而不是直接输出一个连续动作向量。Embedding 的作用是让策略网络能够泛化到相似物品，而不是把推荐问题变成连续动作控制问题。

3.3 REINFORCE 策略梯度

REINFORCE 是一种典型的策略梯度算法。其基本思想是：如果某个动作带来了较高的长期回报，就提高该动作在对应状态下被选择的概率；如果某个动作带来的回报较低，就降低该动作被选择的概率。

对于目标函数

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)],$$

使用 log-trick 可以得到策略梯度：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log \pi_\theta(\tau)].$$

展开到每个时间步，可写为：

$$\nabla_\theta J(\theta) \approx \sum_{\tau \sim \pi_\theta} \sum_{t=0}^{|\tau|} R_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

其中 R_t 是从时间步 t 开始的未来累计回报：

$$R_t = \sum_{t'=t}^{|\tau|} \gamma^{t'-t} r(s_{t'}, a_{t'}).$$

从推荐系统角度看，如果用户在状态 s_t 下被推荐物品 a_t 后，后续产生了较高的观看时长、点击、继续浏览或其他正反馈，那么 R_t 较大，模型就会增大 $\pi_\theta(a_t | s_t)$ 。反之，如果该推荐导致用户快速退出或后续反馈较差，则对应动作概率不会被强化。

因此，该方法中的长期价值主要通过回报 R_t 体现。单步监督排序模型通常只拟合当前点击标签，而 REINFORCE 使用的是未来累计反馈，所以它能够把后续用户行为反向归因到当前推荐动作上。

3.4 为什么需要 Off-Policy Correction

标准 REINFORCE 是 on-policy 方法，要求训练数据来自当前正在优化的策略 π_θ 。但是在工业推荐系统中，这一点通常无法满足。真实日志往往来自历史版本推荐系统，也就是行为策略 β ，而不是当前正在学习的新策略 π_θ 。

如果直接用旧策略产生的数据训练新策略，会产生明显的偏差。例如，旧推荐系统更常推荐热门视频，那么热门视频会获得更多曝光和反馈；如果不进行修正，新策略会继续偏向这些被旧系统频繁曝光的物品，形成“rich get richer”的马太效应。这并不一定说明这些物品本身长期价值最高，而可能只是因为它们被旧策略展示得更多。

为了解决这一问题，论文使用重要性采样进行 off-policy correction。假设轨迹 τ 来自行为策略 β ，则可以将目标策略下的期望改写为：

$$\mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \mathbb{E}_{\tau \sim \beta} \left[\frac{\pi_\theta(\tau)}{\beta(\tau)} R(\tau) \right].$$

其中重要性权重为：

$$\frac{\pi_\theta(\tau)}{\beta(\tau)} = \prod_{t=0}^{|\tau|} \frac{\pi_\theta(a_t | s_t)}{\beta(a_t | s_t)}.$$

完整轨迹级重要性权重方差很大，因此实际中通常使用逐步或近似的重要性修正。对应到每个时间步，可以写成：

$$\nabla_\theta J(\theta) \approx \sum_{\tau \sim \beta} \sum_{t=0}^{|\tau|} \frac{\pi_\theta(a_t | s_t)}{\beta(a_t | s_t)} R_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

其中

$$\frac{\pi_\theta(a_t | s_t)}{\beta(a_t | s_t)}$$

就是 off-policy correction 项。如果某个动作在旧策略下很容易被推荐，但在新策略下概率并不高，那么它的训练权重会被降低；如果某个动作在旧策略下较少被推荐，但新策略认为它有价值，则其权重会被提高。

在真实系统中，行为策略 β 往往不是单一模型，而是多个历史推荐系统或多个推荐模块的混合。因此，论文还训练了一个行为策略估计器来近似 $\beta(a | s)$ ，从而为日志样本提供 propensity correction。

3.5 Top-K 推荐下的特殊修正

上述 off-policy correction 适合单动作推荐，但真实推荐系统通常一次返回 top-K 个物品。如果仍然把每个物品当成独立单动作处理，就会忽略一个事实：推荐系统关心的不是某个物品是否被采样为唯一动作，而是它是否出现在最终 top-K 推荐列表中。

因此，论文提出 top-K off-policy correction。设基础策略为 $\pi_\theta(a | s)$ 。如果从该分布中独立采样 K 次，则物品 a 至少出现一次的概率为：

$$\alpha_\theta(a | s) = 1 - (1 - \pi_\theta(a | s))^K.$$

这里的 $\alpha_\theta(a | s)$ 表示物品 a 进入 top- K 推荐集合的概率。它与单次采样概率 $\pi_\theta(a | s)$ 不同。当 K 较大时，即使 $\pi_\theta(a | s)$ 不需要非常接近 1，该物品也可能有较大概率出现在 top- K 列表中。

将 α_θ 代入策略梯度后，可以得到 top- K 修正后的梯度形式：

$$\nabla_\theta J(\theta) \approx \sum_{\tau \sim \beta} \sum_{t=0}^{|\tau|} \frac{\pi_\theta(a_t | s_t)}{\beta(a_t | s_t)} \lambda_K(s_t, a_t) R_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

其中额外的 top- K 修正因子为：

$$\lambda_K(s_t, a_t) = K (1 - \pi_\theta(a_t | s_t))^{K-1}.$$

该因子的作用非常关键。当

$$\pi_\theta(a_t | s_t) \rightarrow 0$$

时，有

$$\lambda_K(s_t, a_t) \rightarrow K.$$

这说明如果一个高回报物品当前概率很小，top- K 修正会更强地提升它的概率，使其更容易进入推荐列表。

当

$$\pi_\theta(a_t | s_t) \rightarrow 1$$

时，有

$$\lambda_K(s_t, a_t) \rightarrow 0.$$

这说明如果某个物品已经几乎必然进入推荐列表，算法就不再继续把所有概率质量推向该物品。这有助于避免策略把概率过度集中到单个物品上，从而为 top- K 列表中的其他物品保留空间。

因此，top- K correction 的核心直觉是：

推荐系统不需要让一个好物品成为唯一动作，

只需要让它有足够概率进入 top- K 列表。

3.6 训练数据与优化流程

该方法的训练数据来自历史推荐日志。每条轨迹可以表示为：

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots).$$

其中 s_t 是用户状态， a_t 是历史推荐系统在该状态下推荐的物品， r_t 是用户反馈，例如点击、观看时长或其他满意度信号。这些数据并不是当前策略 π_θ 生成的，而是由历史行为策略 β 生成的，因此训练时必须加入 off-policy correction。

整体训练过程可以概括为：

历史日志 $\rightarrow$ 估计用户状态 $\rightarrow$ 计算目标策略概率 $\pi_\theta(a_t | s_t) \rightarrow$ 估计行为策略概率 $\beta(a_t | s_t)$
 $\rightarrow$ 计算回报 $R_t \rightarrow$ 加入 off-policy 与 top-K correction $\rightarrow$ 更新策略网络.

对应的优化目标可以理解为最大化如下加权策略梯度：

$$\sum_{\tau \sim \beta} \sum_{t=0}^{|\tau|} w_t R_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t),$$

其中权重为

$$w_t = \frac{\pi_{\theta}(a_t | s_t)}{\beta(a_t | s_t)} \lambda_K(s_t, a_t).$$

第一项

$$\frac{\pi_{\theta}(a_t | s_t)}{\beta(a_t | s_t)}$$

用于修正新旧策略分布不一致带来的偏差；第二项

$$\lambda_K(s_t, a_t)$$

用于修正 top-K 推荐场景下单物品采样概率与进入列表概率之间的差异。

3.7 该范式的意义

该工作体现了基于 MDP 的长期策略学习范式在推荐系统中的几个关键特点。

首先，它把推荐行为建模为策略 $\pi_{\theta}(a | s)$ ，而不是单纯的监督学习打分函数。模型优化的是长期累计回报，而不是单次曝光下的点击标签。

其次，它通过 REINFORCE 将用户后续反馈 R_t 反向归因到当前推荐动作 a_t ，从而使推荐系统能够学习长期用户价值。这对于短视频、信息流、音乐推荐等连续消费场景尤其重要，因为当前推荐会显著影响用户之后是否继续浏览和再次访问。

再次，它正视了工业推荐系统中的 off-policy 问题。真实训练数据往往来自历史推荐策略，如果不做修正，模型容易复制旧系统的曝光偏差。重要性采样形式的 off-policy correction 可以在一定程度上缓解该问题。

最后，它针对 top-K 推荐提出了专门的修正项。普通 REINFORCE 关注单个动作的采样概率，而推荐系统通常一次展示多个物品。Top-K correction 将优化目标从“让某个物品成为唯一被采样动作”调整为“让高价值物品以合适概率进入推荐列表”，因此更符合真实推荐系统的输出形式。

3.8 小结

总体来看，*Top-K Off-Policy Correction for a REINFORCE Recommender System* 展示了如何将生产级推荐系统建模为 MDP，并使用策略梯度方法直接优化长期用户反馈。其建模方式可以概括为：

用户状态 s_t + 推荐动作 a_t + 用户反馈 r_t + 后续状态 s_{t+1} .

模型学习的是策略

$$\pi_{\theta}(a | s),$$

目标是最大化未来累计回报

$$\sum_t \gamma^t r_t.$$

与传统排序模型相比，该方法的核心区别在于：传统排序模型主要回答“当前用户最可能点击哪个物品”；而基于 MDP 的策略学习回答的是“当前推荐哪个物品，能够带来更高的长期用户收益”。因此，该范式更适合用于长期用户参与度、会话质量、留存和连续消费场景下的推荐优化。

4 基于 Contextual Bandit 的在线探索范式：以 A Contextual-Bandit Approach to Personalized News Article Recommendation 为例

基于 Contextual Bandit 的推荐方法，是强化学习在推荐系统中最早、最经典的一类应用范式。与完整 MDP 强化学习不同，Contextual Bandit 通常不显式建模长期状态转移，而是将推荐过程简化为单步决策问题：系统观察当前用户和候选物品的上下文特征，选择一个物品进行展示，然后根据用户是否点击得到即时奖励，并据此更新推荐策略。

Li 等人在 *A Contextual-Bandit Approach to Personalized News Article Recommendation* 中，将个性化新闻推荐建模为 Contextual Bandit 问题，并提出 LinUCB 算法。该工作面向 Yahoo! 首页新闻推荐场景，其目标是在动态变化的新闻池中，根据用户上下文和新闻内容信息，在线选择最可能被点击的新闻文章。

4.1 问题建模

在新闻推荐场景中，每次用户访问页面时，系统面对一个候选新闻集合

$$\mathcal{A}_t = \{a_{t,1}, a_{t,2}, \dots, a_{t,n}\}.$$

系统需要从候选集合中选择一篇新闻

$$a_t \in \mathcal{A}_t$$

展示给用户。

对于每个候选新闻 a ，系统可以构造一个上下文特征向量

$$x_{t,a} \in \mathbb{R}^d.$$

该特征向量可以包含用户特征、新闻特征以及用户-新闻交叉特征。例如，用户特征可以包括地域、年龄、历史点击兴趣等；新闻特征可以包括类别、主题、发布时间等；交叉特征则用于描述某类用户对某类新闻的偏好。

展示新闻 a_t 后，用户会产生点击或未点击反馈：

$$r_t = \begin{cases} 1, & \text{用户点击新闻,} \\ 0, & \text{用户未点击新闻.} \end{cases}$$

因此，算法的目标是最大化长期累计点击数：

$$\max \sum_{t=1}^T r_t.$$

与 MDP 推荐不同，Contextual Bandit 不考虑当前推荐动作对未来用户状态的影响。也就是说，它关注的是当前上下文下的即时点击收益：

$$\mathbb{E}[r_t \mid x_{t,a}].$$

因此，它更适合新闻推荐、新内容探索、冷启动探索等反馈较快、决策链路较短的场景。

4.2 探索-利用问题

Contextual Bandit 的核心困难是探索-利用平衡。如果系统只推荐当前估计点击率最高的新闻，就可能长期忽略新新闻或冷门新闻，导致模型无法发现潜在高收益内容。如果系统过度随机探索，又会损害用户体验和当前点击率。

因此，推荐策略需要同时考虑两部分：

$$\text{推荐得分} = \text{当前收益估计} + \text{不确定性奖励}.$$

其中，当前收益估计用于利用已有知识，不确定性奖励用于鼓励系统探索估计不充分的新闻。

LinUCB 正是基于这一思想设计的。它假设新闻点击奖励与上下文特征之间近似满足线性关系：

$$\mathbb{E}[r_{t,a} \mid x_{t,a}] = x_{t,a}^T \theta_a,$$

其中 θ_a 是新闻 a 对应的参数向量。由于 θ_a 需要从历史反馈中估计，因此模型对不同新闻的点击收益存在不确定性。LinUCB 使用置信上界来同时表示收益估计和探索价值。

4.3 Disjoint LinUCB 算法

在 Disjoint LinUCB 中，每个新闻文章 a 维护一组独立参数。对于每个候选新闻 a ，维护矩阵

$$A_a \in \mathbb{R}^{d \times d}$$

和向量

$$b_a \in \mathbb{R}^d.$$

其中， A_a 用于累计该新闻相关上下文特征的二阶统计量， b_a 用于累计点击奖励加权后的特征。

参数估计为

$$\hat{\theta}_a = A_a^{-1}b_a.$$

对于当前候选新闻 a ，LinUCB 计算其置信上界分数：

$$p_{t,a} = x_{t,a}^T \hat{\theta}_a + \alpha \sqrt{x_{t,a}^T A_a^{-1} x_{t,a}}.$$

其中，第一项

$$x_{t,a}^T \hat{\theta}_a$$

表示当前模型估计的点击收益；第二项

$$\alpha \sqrt{x_{t,a}^T A_a^{-1} x_{t,a}}$$

表示模型对该估计的不确定性。超参数 α 控制探索强度。当 α 较大时，算法更倾向于探索不确定性较高的新闻；当 α 较小时，算法更倾向于利用当前估计收益较高的新闻。

系统最终选择置信上界分数最大的新闻：

$$a_t = \arg \max_{a \in \mathcal{A}_t} p_{t,a}.$$

当用户反馈 r_t 返回后，只更新被展示新闻 a_t 的参数：

$$A_{a_t} \leftarrow A_{a_t} + x_{t,a_t} x_{t,a_t}^T,$$

$$b_{a_t} \leftarrow b_{a_t} + r_t x_{t,a_t}.$$

未展示的新闻由于没有观察到用户反馈，因此不更新。

4.4 算法流程

Disjoint LinUCB 的流程可以概括如下：

1. 对每个新闻 a 初始化

$$A_a = I_d, \quad b_a = 0.$$

2. 在时刻 t ，观察当前候选新闻集合 $\mathcal{A}_t$ ，并为每个候选新闻 a 构造上下文特征 $x_{t,a}$ 。

3. 对每个候选新闻计算参数估计：

$$\hat{\theta}_a = A_a^{-1}b_a.$$

4. 计算 UCB 推荐分数：

$$p_{t,a} = x_{t,a}^T \hat{\theta}_a + \alpha \sqrt{x_{t,a}^T A_a^{-1} x_{t,a}}.$$

5. 选择分数最高的新闻：

$$a_t = \arg \max_{a \in \mathcal{A}_t} p_{t,a}.$$

6. 展示新闻 a_t ，观察用户点击反馈 r_t 。

7. 更新被展示新闻的参数：

$$A_{a_t} \leftarrow A_{a_t} + x_{t,a_t} x_{t,a_t}^T,$$

$$b_{a_t} \leftarrow b_{a_t} + r_t x_{t,a_t}.$$

4.5 Hybrid LinUCB

Disjoint LinUCB 为每篇新闻维护独立参数，因此不同新闻之间无法共享统计信息。然而在实际新闻推荐中，很多新闻之间存在相似性。例如，同一类别、同一主题或同一热点事件下的新闻，可能具有相似的用户点击模式。如果完全独立建模，每篇新新闻都需要从零开始学习，这会加重冷启动问题。

为了解决这个问题，论文进一步提出 Hybrid LinUCB。Hybrid LinUCB 将奖励期望分解为两部分：一部分是所有新闻共享的全局参数，另一部分是每篇新闻自己的个性化参数：

$$\mathbb{E}[r_{t,a} \mid x_{t,a}, z_{t,a}] = z_{t,a}^T \beta + x_{t,a}^T \theta_a.$$

其中， $z_{t,a}$ 表示用户和新闻之间的共享交叉特征， β 是跨新闻共享的全局参数； $x_{t,a}$ 是新闻 a 自己的上下文特征， θ_a 是新闻 a 的独立参数。

这种结构的好处是：

全局参数负责跨新闻迁移，

局部参数负责刻画单篇新闻的特殊性。

因此，Hybrid LinUCB 更适合动态新闻池和冷启动新闻场景。

4.6 离线评估方法

推荐系统中的在线实验成本较高，因此论文还讨论了如何使用历史随机流量日志对 bandit 算法进行离线评估。核心思想是 replay evaluation。

假设历史日志来自随机推荐策略。每条日志记录为

$$(x_t, \mathcal{A}_t, a_t, r_t),$$

其中 a_t 是历史系统随机展示的新闻， r_t 是用户反馈。对于一个待评估的新算法，在第 t 条日志上，它根据上下文和候选集选择一个动作

$$\hat{a}_t.$$

如果

$$\hat{a}_t = a_t,$$

则说明新算法的选择与历史随机系统当时展示的新闻一致，因此可以使用该条日志的反馈 r_t 来更新和评估新算法。如果

$$\hat{a}_t \neq a_t,$$

则无法知道用户看到 $\hat{a}_t$ 后会不会点击，因此丢弃该条日志。

该方法的关键前提是历史日志来自随机推荐策略，这样保留下来的样本对待评估算法是无偏的。因此，replay evaluation 可以在不上线新算法的情况下，比较不同 bandit 策略的点击表现。

4.7 该算法解决的推荐系统痛点

该工作主要解决了推荐系统中的三个痛点。

首先，它解决了新内容和动态候选池问题。新闻推荐中的文章生命周期很短，候选池不断变化，传统协同过滤很难为新文章积累足够交互数据。LinUCB 可以基于上下文特征进行在线学习，即使新闻刚出现，也可以根据其内容特征和用户特征进行初步推荐。

其次，它解决了探索-利用平衡问题。传统监督排序模型通常只利用历史数据学习点击率，容易偏向已经被大量曝光的热门内容。LinUCB 通过置信上界项显式鼓励探索，使系统有机会发现尚未充分曝光但可能高收益的新闻。

再次，它提供了一种可离线评估的在线学习框架。推荐系统中的在线探索存在风险，直接上线未经验证的策略可能损害用户体验。论文提出的 replay evaluation 利用随机流量日志进行离线评估，为后续工业推荐中的反事实评估和安全上线提供了重要思路。

4.8 与完整强化学习推荐方法的区别

Contextual Bandit 可以看作强化学习推荐系统中的轻量级形式。它与 MDP 强化学习的区别在于：Contextual Bandit 只优化当前上下文下的即时奖励，而不显式建模动作对未来状态的影响。

如果写成强化学习形式，MDP 方法关注的是

$$Q(s_t, a_t) = r_t + \gamma V(s_{t+1}),$$

而 Contextual Bandit 更接近于只关注

$$\mathbb{E}[r_t \mid x_{t,a}].$$

也就是说，Bandit 方法没有显式的长期回报传播，也没有状态转移建模。

因此，Contextual Bandit 更适合反馈即时、决策短链路、探索需求明显的场景；而 MDP 强化学习更适合短视频、信息流、电商会话等长期交互场景。在工业系统中，二者也常常配合使用：Bandit 用于新内容探索、召回探索和冷启动流量分配，MDP 或离线 RL 用于长期用户价值优化。

4.9 小结

总体来看，*A Contextual-Bandit Approach to Personalized News Article Recommendation* 是强化学习思想进入工业推荐系统的经典代表。它将新闻推荐建模为 Contextual Bandit 问题，用 LinUCB 在点击收益估计和不确定性探索之间进行平衡。其核心公式为

$$p_{t,a} = x_{t,a}^T \hat{\theta}_a + \alpha \sqrt{x_{t,a}^T A_a^{-1} x_{t,a}}.$$

其中，第一项表示当前点击收益估计，第二项表示探索价值。

该算法的本质可以概括为：

Contextual Bandit = 上下文建模 + 即时奖励学习 + 不确定性驱动探索。

它不追求完整的长期决策建模，但在新闻推荐、新内容冷启动和在线探索中具有很强的实用价值。因此，在推荐系统的强化学习谱系中，Contextual Bandit 通常被视为最轻量、最容易落地的一类方法。

5 Top-K Off-Policy Correction for REINFORCE 与 SlateQ 对推荐系统落地痛点的解决

强化学习应用到推荐系统时，最大的困难并不只是算法本身，而是推荐系统具有与传统强化学习环境明显不同的工程约束。首先，推荐系统的动作空间极大。动作通常对应具体物品，而工业推荐系统中的候选物品规模可能达到百万级甚至更高。其次，推荐系统通常一次展示多个物品，而不是单个动作，因此推荐动作可能是一个 top- K 列表或 slate，其组合空间规模为

$$\binom{|I|}{K}.$$

再次，真实系统通常不能频繁在线试错，训练数据大多来自历史推荐日志，这会带来明显的 off-policy 偏差。最后，推荐系统不仅关心当前点击，还关心长期观看、留存、复访和用户满意度。因此，强化学习推荐系统需要同时解决

超大离散动作空间 + 离线日志偏差 + top- K 多物品输出 + 长期价值优化

这几个问题。

Top-K Off-Policy Correction for REINFORCE 和 SlateQ 分别从 policy-based 和 value-based 两条路线解决这些痛点。前者的重点是让 REINFORCE 能够在生产级 top- K 推荐系统中使用；后者的重点是让 Q-learning 能够处理 slate/list 这种组合动作空间。

5.1 Top-K Off-Policy Correction for REINFORCE 解决的问题

Top-K Off-Policy Correction for REINFORCE 将推荐系统建模为 MDP，并学习一个随机策略

$$\pi_{\theta}(a \mid s),$$

其中 s 表示用户状态， a 表示候选物品。策略的目标是最大化长期回报：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

与传统监督排序不同，REINFORCE 不是只根据当前点击标签更新模型，而是使用未来累计回报 R_t 更新当前推荐动作：

$$\nabla_{\theta} J(\theta) \approx \sum_t R_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t).$$

因此，如果某个推荐物品不仅带来了当前点击，还带来了后续观看、继续浏览或长期参与度提升，其对应动作概率就会被增强。

该方法首先解决的是**长期价值建模问题**。传统 CTR 排序模型学习的是

$$P(\text{click} \mid s, a),$$

而 REINFORCE 学习的是

$$\pi_{\theta}(a \mid s)$$

使得长期累计反馈最大。也就是说，它回答的不是“当前用户最可能点击哪个物品”，而是“当前推荐哪个物品更有利于长期用户收益”。

其次，该方法解决的是**离线日志带来的 off-policy 偏差**。标准 REINFORCE 要求训练轨迹来自当前策略 π_{θ} ，但工业推荐系统中的日志通常来自历史策略或多个旧推荐器，记为行为策略 β 。如果直接用这些日志训练当前策略，模型会继承历史策略的曝光偏差。例如，热门物品被历史系统频繁曝光，因此日志中有更多反馈；如果不修正，模型可能把“被展示得多”误认为“长期价值高”。

为此，该方法引入重要性采样形式的 off-policy correction:

$$w_t = \frac{\pi_{\theta}(a_t \mid s_t)}{\beta(a_t \mid s_t)}.$$

修正后的策略梯度可以写成

$$\nabla_{\theta} J(\theta) \approx \sum_t w_t R_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t).$$

其中 $\beta(a_t \mid s_t)$ 表示历史策略在状态 s_t 下推荐物品 a_t 的概率。该项的作用是修正目标策略和日志策略之间的分布差异，使模型不只是复制旧系统的曝光偏置。

第三，该方法解决的是**top-K 推荐与单动作策略不一致的问题**。普通 REINFORCE 假设每次只采样一个动作，但真实推荐系统往往一次返回 K 个物品。因此，系统关心的不是某个物品是否成为唯一动作，而是它是否能够进入最终的 top-K 推荐列表。

如果单物品策略概率为

$$\pi_{\theta}(a \mid s),$$

那么在一种简化管理下，物品 a 至少进入 K 次采样结果中的概率可以写成

$$\alpha_K(a \mid s) = 1 - (1 - \pi_{\theta}(a \mid s))^K.$$

其中 $\alpha_K(a \mid s)$ 表示物品进入 top-K 推荐集合的概率。于是，top-K 场景下更合理的优化对象不再是

$$\log \pi_{\theta}(a \mid s),$$

而是

$$\log \alpha_K(a \mid s).$$

对应地，梯度会根据物品进入 top-K 的概率进行修正。其直观含义是：

推荐系统不需要把一个高价值物品变成唯一最高概率动作，

只需要让它以合适概率进入 top- K 列表。

这可以避免策略把全部概率质量过度集中到少数物品上，更符合真实推荐系统一次展示多个物品的输出形式。

第四，该方法缓解了**超大离散动作空间问题**。在推荐系统中，动作虽然可以用 embedding 表示，但动作本质仍然是离散物品：

$$a \in \mathcal{I}.$$

物品 embedding 的作用是参数化策略，例如通过用户状态向量 h_s 和物品向量 v_a 计算匹配分数：

$$\pi_{\theta}(a | s) \propto \exp(h_s^T v_a).$$

这样，策略网络不需要为每个物品单独学习完全独立的参数，而是可以通过 embedding 在相似物品之间泛化。在实际系统中，通常还会结合候选采样、sampled softmax 或召回候选集，避免每次在全量百万级物品上计算完整 softmax。

因此，Top-K Off-Policy Correction for REINFORCE 的解决路线可以概括为：

参数化离散动作策略 + 历史日志 off-policy 修正 + top- K 曝光概率修正 + 长期回报优化。

它并没有把离散动作空间变成连续动作空间，而是用 embedding 和采样机制让超大离散动作空间变得可训练。

5.2 SlateQ 解决的问题

SlateQ 面对的是另一类更严重的动作空间问题：推荐系统通常一次展示的是一个列表

$$A = \{i_1, i_2, \dots, i_K\},$$

如果直接把整个列表当作动作，则动作空间规模为

$$\binom{|\mathcal{I}|}{K}.$$

当 $|\mathcal{I}|$ 很大时，直接学习

$$Q(s, A)$$

几乎不可行。这就是 slate recommendation 中的组合动作空间爆炸问题。

SlateQ 的核心思想是：不直接学习整个列表的 Q 值，而是把列表价值分解成用户可能选择的单个物品的长期价值。设用户在状态 s 下看到列表 A 后选择物品 i 的概率为

$$P(i | s, A),$$

物品 i 被用户消费后的长期价值为

$$Q(s, i).$$

则 slate-level 价值可以分解为

$$Q(s, A) = \sum_{i \in A} P(i | s, A) Q(s, i).$$

这个公式将组合动作价值

$$Q(s, A)$$

转化为 item-level 价值

$$Q(s, i)$$

的加权和。因此，SlateQ 不需要对所有可能列表分别学习 Q 值，而只需要学习每个物品在当前用户状态下的长期价值。

这一步解决了**组合动作空间爆炸**。原本需要处理的动作数量是

$$\binom{|\mathcal{I}|}{K},$$

而分解之后，模型主要学习的是

$$Q(s, i), \quad i \in \mathcal{I}.$$

也就是说，SlateQ 把列表级 RL 问题转化成了物品级长期价值估计和列表级组合优化问题。

SlateQ 还通过用户选择模型处理列表内部的竞争关系。在 MNL 用户选择模型下，设物品吸引力为

$$v(s, i) > 0,$$

用户不点击任何物品的吸引力为

$$v(s, \perp) > 0.$$

则用户选择物品 i 的概率为

$$P(i | s, A) = \frac{v(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

代入 SlateQ 分解式可得：

$$Q(s, A) = \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

这个式子说明，列表价值不是简单的

$$\sum_{i \in A} Q(s, i),$$

也不是简单的

$$\sum_{i \in A} v(s, i) Q(s, i).$$

因为每个物品进入列表后，不仅会增加分子中的长期收益贡献，也会增加分母中的总吸引力，从而影响其他物品被点击的概率。因此，SlateQ 显式建模了列表中物品之间的点击竞争关系。

在求解最优列表时，SlateQ 可以把问题写成：

$$A^* = \arg \max_{A \subseteq \mathcal{I}, |A|=K} \frac{\sum_{i \in A} v_i Q_i}{v_0 + \sum_{i \in A} v_i},$$

其中

$$v_i = v(s, i), \quad Q_i = Q(s, i), \quad v_0 = v(s, \perp).$$

该问题虽然仍然是组合优化问题，但它已经不需要枚举所有 slate。对于固定阈值 λ ，可以转化为选择

$$v_i(Q_i - \lambda)$$

最大的 K 个物品。随后根据当前列表价值更新 λ ：

$$\lambda \leftarrow \frac{\sum_{i \in A} v_i Q_i}{v_0 + \sum_{i \in A} v_i}.$$

通过这种迭代方式，SlateQ 将原本不可枚举的列表优化问题，转化为若干次 top- K 选择问题。在实际系统中，通常先由召回模型产生较小候选集 $\mathcal{C}$ ，再在 $\mathcal{C}$ 上进行 SlateQ 列表优化：

$$A^* = \arg \max_{A \subseteq \mathcal{C}, |A|=K} Q(s, A).$$

因此，SlateQ 对推荐系统落地痛点的解决可以概括为：

item-level 长期价值学习 + 用户选择模型 + slate 价值分解 + 可求解的列表优化。

它解决的重点不是单个物品动作空间大，而是整个推荐列表作为组合动作时无法直接进行 Q-learning 的问题。

5.3 二者对超大离散动作空间的处理差异

Top-K Off-Policy Correction for REINFORCE 和 SlateQ 都面对推荐系统中的超大动作空间，但它们处理问题的角度不同。

Top-K REINFORCE 的动作仍然是单个物品：

$$a \in \mathcal{I}.$$

它通过参数化策略

$$\pi_{\theta}(a | s)$$

为每个物品分配被推荐概率。由于物品数量巨大，实际训练中依赖 item embedding、候选采样和近似 softmax 等方法，使得百万级离散动作空间可以被策略网络处理。同时，它通过 top- K correction 把单物品采样概率修正为进入 top- K 列表的概率，从而适配真实推荐系统一次输出多个物品的形式。

SlateQ 的动作则是一个列表：

$$A \subseteq \mathcal{I}, \quad |A| = K.$$

如果直接学习 $Q(s, A)$ ，动作空间是组合级别的。SlateQ 通过

$$Q(s, A) = \sum_{i \in A} P(i | s, A) Q(s, i)$$

把 slate-level 价值分解为 item-level 价值，从而避免直接枚举所有推荐列表。它解决的是组合动作空间爆炸问题，而不是单纯的单物品动作空间过大问题。

二者的区别可以总结如下：

方法	动作建模	解决动作空间问题的方式
Top-K REINFORCE	单物品动作 a	embedding 参数化策略 + 采样近似 + top- K 修正
SlateQ	列表动作 A	slate 价值分解 + 用户选择模型 + top- K 列表优化

更直观地说，Top-K REINFORCE 解决的是：

如何在百万级物品中学习一个能产生 top- K 推荐的策略。

SlateQ 解决的是：

如何避免把每一个推荐列表都当成一个独立动作来学习 Q 值。

5.4 二者分别适合的推荐场景

如果系统更关注策略学习本身，例如希望直接优化长期观看、长期点击或用户参与度，并且训练数据主要来自历史日志，那么 Top-K Off-Policy Correction for REINFORCE 更合适。它适合 feed、短视频、音乐推荐等连续消费场景，尤其适合需要从旧策略日志中学习新策略的工业系统。

如果系统更关注列表级重排，例如希望在候选集上选择一个整体最优的推荐列表，同时考虑物品之间的竞争、点击分配和长期价值，那么 SlateQ 更合适。它适合精排后的重排、相关推荐列表、首页模块列表等 slate recommendation 场景。

两者并不是互斥关系。在一个完整推荐系统中，可以先用召回和排序模型产生候选集，再用类似 Top-K REINFORCE 的策略学习方法优化长期用户反馈，也可以在最后重排阶段使用 SlateQ 对候选列表进行长期价值优化。其共同目标都是让推荐系统从短期点击优化走向长期决策优化。

5.5 小结

Top-K Off-Policy Correction for REINFORCE 和 SlateQ 分别代表了强化学习推荐系统中的两条重要路线。Top-K REINFORCE 属于 policy-based 方法，重点解决生产日志下的 off-policy 学习、超大离散物品空间和 top- K 输出问题；SlateQ 属于 value-based 方法，重点解决 slate/list 推荐中组合动作空间爆炸和列表级长期价值评估问题。

因此，可以将二者的核心贡献概括为：

Top-K REINFORCE：让策略梯度能在生产级 top- K 推荐中训练。

SlateQ：让 Q -learning 能在组合型推荐列表动作中可解。

从推荐系统落地角度看，Top-K REINFORCE 更像是在解决“如何从历史日志中训练一个长期收益更高的推荐策略”；SlateQ 更像是在解决“如何对一个推荐列表整体计算长期价值并进行可扩展优化”。二者共同说明，强化学习推荐系统的关键不是把 item embedding 当成连续动作，而是在超大离散动作和组合列表动作下，通过参数化、采样、off-policy 修正和价值分解等方法，让长期决策优化变得可训练、可评估、可上线。

6 Contextual-Bandit新闻推荐方法总结

Li等人在论文 *A Contextual-Bandit Approach to Personalized News Article Recommendation* 中, 将个性化新闻推荐问题建模为上下文多臂老虎机问题 (contextual bandit) [?]. 传统协同过滤方法通常依赖较稳定的用户-物品交互历史, 但新闻推荐具有内容更新快、候选文章池动态变化明显、冷启动频繁等特点, 因此仅依赖历史点击共现关系难以有效处理新文章和新用户场景。同时, 线上推荐系统需要在大规模流量下快速学习和实时决策, 这使得推荐算法不仅要具有较好的点击率表现, 还要具备较低的计算复杂度。

在该问题中, 每篇候选新闻文章被视为一个动作或臂。在第 t 次推荐时, 系统观察当前用户 u_t 以及候选文章集合 $\mathcal{A}_t$, 并为每个候选文章 $a \in \mathcal{A}_t$ 构造上下文特征 $x_{t,a}$ 。该特征可以同时包含用户信息、文章信息以及二者的交互信息。系统根据历史反馈选择一篇文章 a_t 展示给用户, 随后只能观察该文章是否被点击, 即奖励

$$r_{t,a_t} = \begin{cases} 1, & \text{用户点击文章,} \\ 0, & \text{用户未点击文章.} \end{cases}$$

对于未被展示的文章, 系统无法知道用户是否会点击, 这正是 bandit 问题区别于普通监督学习推荐问题的关键。算法目标是在长期交互过程中最大化累计点击数, 等价于最小化与最优推荐策略之间的累计遗憾。

论文提出的核心算法是 LinUCB。其基本思想是: 一方面利用已有点击数据估计每篇文章的期望点击率, 另一方面利用置信上界刻画模型不确定性, 从而在 exploitation 和 exploration 之间进行平衡。在 disjoint linear model 中, 假设每篇文章 a 有独立的线性参数 θ_a , 其期望奖励满足

$$\mathbb{E}[r_{t,a} | x_{t,a}] = x_{t,a}^\top \theta_a.$$

根据历史样本, 算法使用岭回归估计文章 a 的参数:

$$\hat{\theta}_a = A_a^{-1} b_a,$$

其中

$$A_a = I + \sum x_{t,a} x_{t,a}^\top, \quad b_a = \sum r_{t,a} x_{t,a}.$$

在推荐时, LinUCB 为每个候选文章计算如下置信上界分数:

$$p_{t,a} = x_{t,a}^\top \hat{\theta}_a + \alpha \sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}},$$

并选择分数最高的文章:

$$a_t = \arg \max_{a \in \mathcal{A}_t} p_{t,a}.$$

其中, 第一项表示当前模型对点击率的估计, 第二项表示不确定性奖励。当某篇文章或某类上下文样本较少时, 其不确定性较大, 算法会更倾向于探索; 当数据逐渐充分后, 不确定性下降, 算法会更多地利用当前认为点击率较高的文章。

除 disjoint LinUCB 外, 论文还提出了 hybrid LinUCB。该模型同时包含全局共享参数和文章独立参数:

$$\mathbb{E}[r_{t,a} | z_{t,a}, x_{t,a}] = z_{t,a}^\top \beta + x_{t,a}^\top \theta_a.$$

其中, β 是所有文章共享的参数, 用于捕捉不同文章之间可迁移的用户-文章交互规律; θ_a 是文章 a 独有的参数, 用于刻画单篇文章自身的点击偏好。相比 disjoint 模型, hybrid 模型能够在不同文章之间共享统计信息, 因此对于新文章、稀疏样本和冷启动场景更加友好。

论文的另一个重要贡献是提出了基于随机日志的离线评估方法。由于 bandit 推荐只能观察被展示文章的反馈, 普通离线评估容易产生选择偏差。论文使用历史随机曝光日志进行 replay evaluation: 当待评估算法在某一步选择的的文章与日志中真实随机展示的文章一致时, 才使用该条样本更新算法并记录奖励; 否则跳过该样本。在日志展示策略随机且样本独立同分布的条件下, 这种方法可以得到对目标 bandit 策略的无偏估计, 从而避免直接在线试错带来的用户体验风险。

实验基于 Yahoo! Front Page Today Module 的大规模新闻推荐数据集, 包含超过 3300 万条事件日志。实验结果表明, 引入上下文特征的算法明显优于不使用用户和文章特征的 context-free bandit 方法。其中 LinUCB 在多个设置下取得了更高的相对 CTR, 相较普通 ϵ -greedy 等基线方法具有明显提升。论文报告中, LinUCB 相比标准 context-free bandit 方法最高取得了约 12.5% 的点击提升, 并且在训练数据较少时, 上下文信息带来的优势更加明显。

总体来看, 该论文的主要意义在于: 它将新闻推荐从静态预测问题转化为在线序贯决策问题, 明确刻画了推荐系统中的探索-利用矛盾; 同时, LinUCB 通过线性模型和置信上界机制, 在计算效率、在线学习能力和个性化效果之间取得了较好的平衡。这篇工作也是 contextual bandit 在推荐系统中落地应用的经典代表, 为后续广告推荐、内容推荐、冷启动推荐以及在线策略评估等方向提供了重要基础。